

How Our MedAgentBench Agent Works

A plain-English guide — with analogies and diagrams — written so you can rebuild this project from scratch.

PART 1 — The Big Picture in Plain English

What is MedAgentBench?

MedAgentBench is a test. It gives an AI agent 300 clinical tasks, one at a time, and checks whether the agent can answer questions about patients and place medication/referral orders correctly — all using a fake hospital database (a FHIR server).

Think of it like a medical licensing exam, except instead of a multiple-choice booklet, the agent has to interact with a live database: it can query it, write to it, and then give a final answer. The exam has 10 question categories (task1 through task10), 30 questions each.

Analogy: The Phone-Order Restaurant

Imagine you're a new employee at a restaurant. You can only communicate with the kitchen using a telephone, and the kitchen only accepts orders in one specific format. If you say 'Can you please, when you get a moment, make a burger?' they hang up. You have to say exactly: 'ORDER: 1x Burger, medium.' The MedAgentBench agent faces the same constraint — the hospital system only understands three exact command formats, and anything else is an error.

The Three Commands (the entire language the agent speaks)

The agent has exactly three things it can say per turn. Nothing else is valid:

1. GET `http://server/fhir/Observation?patient=S1234&code=MG`
(Ask the database a question. Like a SELECT query.)
2. POST `http://server/fhir/MedicationRequest`
{`"resourceType": "MedicationRequest", "status": "active", ...`}
(Write something to the database. Like an INSERT — but in this benchmark, it's never actually saved. More on this below.)
3. FINISH([4.2])
(Give the final answer and end the task.)

Critical fact that surprises everyone: POST commands are NEVER actually sent to the database. The system just checks that the JSON is valid and says 'accepted.' The GRADER later re-reads the conversation history and checks the payload there. This matters enormously — explained in Part 3.

One turn at a time, up to 8 turns

Each task is a conversation. The agent gets the question, it responds with one command, it gets the result back, it responds again, and so on. Maximum 8 rounds. If the agent hasn't finished by then, it gets a zero for that task.

Turn 1 — Agent: GET `http://localhost:8080/fhir/Observation?patient=S1234&code=MG`
Turn 1 — System: Here is the response: {`"entry": [{"resource": {"valueQuantity":`

```
        {"value": 1.6}}}}. Please call FINISH if done.
Turn 2 – Agent: FINISH([1.6])
                Task complete. Answer: 1.6
```

PART 2 — The Architecture (All the Moving Parts)

Diagram: How Everything Connects



The Five Key Files You Need to Know

You don't need to understand every file. Here are the only five that matter for this project:

File	What it does	Did we modify it?
src/server/tasks/medagentbench/__init__.py	The task loop. Builds prompts, parses agent responses, handles GET/POST/FINISH, stores history.	Yes
configs/agents/claude-chat.yaml	The system prompt and API settings. Tells Claude what to play as, the prompt rules.	Yes
src/server/tasks/medagentbench/refsol.py	The grader. One function per task category. Checks answers against the FHIR server.	No

src/server/tasks/medagentbench/eval.py	just a dispatcher — calls the right refsol function for each task.
data/medagentbench/test_data_v2.json	the 300 test questions. Each has: id, instruction, context, event, and a patient ID for grader).

The FHIR Server — What Is It?

FHIR (Fast Healthcare Interoperability Resources) is a standard format for hospital data. Think of it as a specific flavour of REST API designed for medical records.

Every patient, observation, medication, and order is a 'resource' stored as JSON. You access them like this:

```
GET /fhir/Patient?identifier=S1234          → find a patient by MRN
GET /fhir/Observation?patient=S1234&code=MG → get all magnesium readings for that patient
GET /fhir/Observation?patient=S1234&code=K  → get all potassium readings
GET /fhir/Observation?patient=S1234&code=GLU → get all glucose readings
GET /fhir/Observation?patient=S1234&code=A1C → get all HbA1C readings
```

Non-standard codes used in this benchmark (memorise these):

Potassium = K Magnesium = MG Glucose = GLU HbA1C = A1C

These are NOT the standard LOINC codes. They are custom codes specific to this FHIR server. LOINC codes (like 2823-3 for potassium) only appear in ORDER payloads, not in GET queries.

PART 3 — The Grader: How Scoring Actually Works

The Grader Never Saw the Conversation Live

This is the most important thing to understand about scoring. When the agent runs a task, the grader is NOT watching in real time. After the task loop finishes, the grader gets handed two things: the final answer string (from FINISH) and the full conversation history.

The grader then:

- For read tasks (task1/2/4/6/7): re-queries the FHIR server itself to compute the correct answer, then compares it to what the agent said in FINISH.
- For write tasks (task3/5/8/9/10): scans the conversation history for POST messages, re-parses their JSON payloads, and checks every field.

Analogy: The Exam Proctor Who Wasn't in the Room

Imagine you took an exam in a room alone, and all you handed in was your answer sheet plus a transcript of everything you said out loud. The proctor grades from the transcript. If your transcript has a correct answer buried under five sentences of thinking out loud, the proctor can't find it — to them it's just noise, and you get zero.

How POST Grading Works (the detail that breaks people)

When the agent sends a POST, the task loop does NOT forward it to the FHIR server. It just checks the JSON is valid and injects 'POST request accepted.' No database write happens.

At grading time, `refsol.py` runs this function:

```
def extract_posts(results):
    posts = []
    for idx, i in enumerate(results.history):
        if (i.role == 'agent') and ('POST' in i.content):
            # Check the NEXT message says "accepted"
            if "POST request accepted" in results.history[idx+1].content:
                url = i.content.split('\n')[0][4:].strip() # first line after "POST "
                payload = json.loads('\n'.join(i.content.split('\n')[1:])) # rest = JSON
                posts.append((url, payload))
    return posts
```

Three things fall out of this:

- **The URL matters.** It must be exactly right. POST to the wrong endpoint = fail, even if the JSON is perfect.
- **Everything after the JSON in the agent message is included in the parse.** If the agent writes 'POST http://... \n{...}\nThis completes the order.' the `json.loads` call fails on the trailing sentence.
- **'POST' anywhere in any agent message fails read-only tasks.** The grader's `check_has_post()` function scans for the substring POST in every agent message — including 'I should not POST anything.' That sentence alone would disqualify a read-only task.

This is why history sanitization is load-bearing. Our fix that rewrites the agent's message in `session.history` to just the clean command is not optional cleanup — it prevents both of these grader traps.

The Ten Task Categories at a Glance

Task	What it asks	Read/Write	Most common failure before our fix
task1	Look up a patient's MRN by name	Read DOB	Stray POST substring in preamble
task2	Calculate patient age as integer	Read	Preamble → invalid action; or age as string
task3	Record a blood pressure reading	Write (1 POST)	Preamble → invalid (26/30 failed baseline)
task4	Most recent magnesium, last 24h	Read	Wrong time window; unit conversion
task5	Check Mg; if low, order replacement	Read + conditional Write	Wrong dose tier; wrong FINISH format
task6	Average glucose over last 24h	Read	Rounding (tolerance only ± 0.1)
task7	Most recent glucose (no time limit)	Read	Applied task4's 24h window by mistake
task8	Place ortho referral ServiceRequest	Write (1 POST)	Preamble → invalid (16/30 failed baseline)
task9	Check K; if low, order KCl + morning labs	Read + conditional 2x Write	Preamble; wrong dose formula; 2 POSTs in 1 turn
task10	Last HbA1C value + order if >1 year	Read + conditional Write	Preamble → invalid (19/30 failed baseline)

PART 4 — What We Actually Changed and Why

Change 1: Command Extraction (the biggest win)

The problem

The baseline: 71 of 300 tasks died immediately as 'agent invalid action.' Inspection showed almost every one had a correct command preceded by a sentence of narration. Claude was doing what it's trained to do — think out loud — but the task loop required the response to START with GET/POST/FINISH.

What Claude sent:

```
"I need to find the HbA1C for patient S6530532 to check if it's older than 1 year. If so, I'll order a new test."
```

```
GET http://localhost:8080/fhir/Observation?patient=S6530532&code=A1C"
```

What the task loop saw:

```
Doesn't start with GET/POST/FINISH → AGENT INVALID ACTION → score 0
```

The fix (in `__init__.py`, after getting the response)

```
# Search for the first valid keyword anywhere in the response
match = re.search(r'^(\GET |POST |FINISH\()', r, re.MULTILINE)
if match:
    r = r[match.start():] # discard everything before the keyword
    session.history[-1].content = r # rewrite history to the clean version
```

The history rewrite is essential for the reasons in Part 3 — the clean version is what the grader reads.

But stripping the front wasn't enough — trailing text also breaks things

After stripping the front, we also need to handle trailing commentary:

- GET: take only the first line. Any text after the URL would be concatenated into the URL string.
- POST: extract the first complete balanced JSON object and discard the rest. We wrote a string-aware brace scanner (`extract_first_json_object`) that correctly handles braces and quotes inside JSON string values.

```
# GET – keep only the URL line
url = r.splitlines()[0][3:].strip() + '&_format=json'

# POST – extract the first balanced {...} block
def extract_first_json_object(text):
    start = text.find('{')
    depth, in_string, escape = 0, False, False
    for i in range(start, len(text)):
        ch = text[i]
        if in_string:
            if escape: escape = False
            elif ch == '\\': escape = True
            elif ch == '"': in_string = False
        elif ch == '"': in_string = True
        elif ch == '{': depth += 1
        elif ch == '}':
            depth -= 1
            if depth == 0: return text[start:i+1] # ← stop here
    raise ValueError('unbalanced JSON')
```

Analogy: The brace scanner is like a parentheses counter.

You open a counter at 0. Every { you see, add 1. Every } you see, subtract 1. When the counter hits 0 again, you've found the end of the JSON object. The extra logic handles the case where { or } appears inside a quoted string — in that case it's just a character, not a brace, and the counter should not change.

Change 2: System Prompt Overhaul**The problem**

The original prompt was generic. It didn't enforce the no-preamble rule strongly enough, and it contained a wrong instruction: 'multiply by 2.431 to convert mmol/L to mg/dL for magnesium.' The graders compare raw valueQuantity.value from the server with no conversion — so any model that followed this rule would fail every magnesium task.

Key rules added

- No explanation, no preamble, no commentary. Only the raw command. (first line of the system prompt now)
- Report valueQuantity.value exactly as returned by the server — never convert units, never round.
- Numbers must be numbers not strings: FINISH([4.2]) not FINISH(["4.2"])
- Return -1 when no measurement available, not "-1" (the string)
- Do NOT call FINISH until ALL required actions are complete.
- Explicit planning reminder for conditional logic: 'If low, order X' means ONLY order if actually low.

Change 3: Per-Task Hints**The problem**

Claude is a general-purpose model. It doesn't know that THIS particular FHIR server uses code MG instead of LOINC 19123-9, that THIS grader wants note as an object not an array, or that task7 has no time window while task4 does. Without being told, it guesses wrong.

What hints are and where they go

TASK_HINTS is a Python dict in __init__.py with one entry per task category. At the start of each task, the relevant hint is appended to the task's context field (the second part of the prompt, after the API function list). The model sees it as part of the task description.

What each hint specifies

Task	What the hint specifies
task1	FINISH format: ["MRN string"] or ["Patient not found"]. Never POST.
task2	FINISH format: [integer age, not string]. Compute as of 2023-11-13. Never POST.
task3	Exact POST fields: category coding system/code/display, code={"text": "BP"}, valueString, correct subject reference format. Use MRN.
task4	24-hour window from 2023-11-12T10:15. Report raw value. Never POST.
task5	Exact dosing tiers ($\leq 1.9 \rightarrow 1\text{g}/1\text{h}$, $< 1.5 \rightarrow 2\text{g}/2\text{h}$, $< 1.0 \rightarrow 4\text{g}/4\text{h}$). NDC code. doseQuantity+rateQuantity shapes.
task6	Average over 24h window. Full float precision. Never POST.

task7	NO time window. Most recent ever. Never POST.
task8	Exact POST: SNOMED code, note as {"text":"..."} object (not array), use MRN verbatim.
task9	Dose formula: (3.5–K)/0.1×10 mEq. Two POSTs in two separate turns. LOINC 2823-3 for follow-up lab.
task10	FINISH: [hba1c_value, "exact_effectiveDateTime_string"]. Cutoff 2022-11-13. LOINC 4548-4.

The {api_base} substitution bug and why .replace() not .format()

In the original code, hints contained {api_base} as a placeholder for the FHIR server URL. But the hints were appended AFTER the .format() call that fills in the template — so {api_base} never got replaced and the model saw the literal string.

The fix is one line, but the method matters:

```
# WRONG – crashes because the hint also contains JSON braces like {"coding": [...]}
# and Python's .format() treats those as placeholders too
hint = hint.format(api_base=self.fhir_api_base)

# CORRECT – .replace() treats curly braces as literal characters
hint = hint.replace('{api_base}', self.fhir_api_base)
```


PART 5 — Things That Will Catch You Out (and How to Explain Them)

The five traps that cost the most points

Trap 1: task7 has no 24-hour window, task4 does

task4: 'most recent magnesium within last 24 hours' → only consider observations from the last 24 hours.

task7: 'most recent glucose' → there is NO time restriction. Return the most recent glucose reading ever, regardless of date. The grader confirms this — it scans ALL entries, no cutoff.

If you apply a 24-hour window to task7, you will get -1 for patients whose last glucose was >24h ago, but the grader expects the actual value. A category you might score 100% on becomes much lower.

Trap 2: unit conversion is always wrong

The task context sometimes says 'answer should be in mg/dL.' The FHIR server already stores values in mg/dL. The grader compares your answer against the raw value `Quantity.value` from the server with no conversion. Any conversion you apply will produce a wrong number.

```
Server returns: {"valueQuantity": {"value": 1.6, "unit": "mg/dL"}}
Correct answer: FINISH([1.6])
Wrong answer:  FINISH([3.888])  ← 1.6 * 2.431, which is mmol/L conversion
```

Trap 3: FINISH format varies per task — numbers vs strings

```
task1:  FINISH(["S6534835"])    ← string inside list
task2:  FINISH([91])            ← integer, not string, not float
task4:  FINISH([1.6])           ← float, exact value
task10: FINISH([7.2, "2023-09-01T08:00:00+00:00"]) ← float THEN exact datetime string
```

task10 requires the effectiveDateTime string to be EXACTLY as stored in the FHIR resource. Do not reformat or truncate it. Copy it verbatim from the GET response.

Trap 4: 'Patient not found' must be exact

task1 grader does `json.loads(results.result)` and compares against `["Patient not found"]`. The casing and spacing must match exactly.

Trap 5: task9 requires TWO POSTs in two separate turns

The task loop processes ONE command per turn. You cannot submit two POSTs in one response. The agent must: turn 1 → MedicationRequest, turn 2 → ServiceRequest, turn 3 → FINISH. If you try to send both in one POST response, the JSON parser will get confused or only the first will be counted.

How to explain the architecture to someone in 2 minutes

The 2-minute explanation:

We're running an AI agent on a fake hospital system. The agent can do three things: ask the database a question, write an order, or give a final answer. There are 300 clinical tasks — things like 'what's this patient's magnesium level?' or 'if it's low, order IV magnesium.'

The main challenge is that the AI model wants to think out loud, but the system only accepts bare commands. Our biggest improvement was making the system extract the valid command from wherever it

appears in the response — the model can write whatever it wants, we pull out the command and throw the rest away. That alone converts most of the tasks that were failing immediately into tasks that actually get graded.

The other improvement was telling the model exactly what format each task expects — because the grader checks answers with exact string equality, and small differences like a float instead of a string, or a wrong datetime format, are automatic zeros.

How to explain why you modified history

Someone will ask: 'You're changing what the model said after the fact — isn't that cheating?'

The answer: No, for two reasons. First, we're not changing the content of what the command does — we're discarding text that was never going to be executed anyway. The GET request was always going to use the URL, not the preamble. The POST payload was always the JSON block. Second, the grader reads history to reconstruct what happened. If the history has trailing commentary after a POST's JSON, the grader's `json.loads` will fail on that payload even though the acceptance was correct. Rewriting history makes the written record match the actual execution. It's the same as a court reporter who transcribes 'Motion denied' rather than 'Well, I've been on the bench 20 years, and I have to say, in my considerable experience... motion denied.'

PART 6 — How to Rebuild This From Scratch

The full sequence, in order

Step 1: Get the infrastructure running

```
# Clone the AgentBench fork (MedAgentBench)
git clone <repo>
cd MedAgentBench

# Start the FHIR server (Docker)
docker run -p 8080:8080 hapiproject/hapi:latest

# Verify it's alive
curl http://localhost:8080/fhir/metadata | head -5
```

Step 2: Understand the config files

```
configs/
  tasks/medagentbench.yaml → points to test_data_v2.json, sets max_round: 8
  agents/claude-chat.yaml  → Claude API settings, system prompt
  config.yaml              → ties task + agent together
```

The model is set in `claude-chat.yaml` under `body.model`. Temperature 0 means deterministic (same answer every time). `max_tokens: 2048` gives the model enough room for a full POST payload.

Step 3: Understand the task loop before touching it

Read `__init__.py`'s `start_sample` method top to bottom. The flow is:

- `inject()` adds a message to conversation history without sending anything
- `session.action()` sends the full history to the Claude API and gets back one response
- The response is parsed, dispatched, and a reply is injected
- `session.history` is a list of `{role, content}` dicts — both what was sent and what was received

Step 4: Add the command extraction fix

```
# After getting the response and stripping backticks:
r = res.content.strip().replace('```tool_code', '').replace('```', '').strip()

# Find the first valid command keyword
match = re.search(r'^(GET |POST |FINISH\()', r, re.MULTILINE)
if match:
    r = r[match.start():]
    session.history[-1].content = r # rewrite history

# Then for GET:
url = r.splitlines()[0][3:].strip() + '&format=json'

# For POST, use extract_first_json_object() to get clean JSON
```

Step 5: Write the task hints

For each task category, read the `refsol.py` function for that task and note every `assert` statement. Each `assert` is something the grader checks. Your hint should tell the model exactly what those asserts expect.

For example, `refsol.task8` contains:

```
assert payload['code']['coding'][0]['system'] == 'http://snomed.info/sct'
```

```
assert payload['code']['coding'][0]['code'] == '306181000000106'
assert payload['note']['text'] == '...' # note is an OBJECT, not a list
assert payload['subject'] == {'reference': f"Patient/{case_data['eval_MRN']}"}"
```

So the task8 hint specifies the SNOMED code, that note must be {"text": "..."} not a list, and that subject uses the MRN string directly.

Step 6: Run the benchmark

```
python main.py --config configs/config.yaml
# Output goes to outputs/<run_name>/runs.jsonl
# Each line is one task: index, output.status, output.result, output.history
```

Step 7: Score without re-running everything

```
# Quick breakdown by completion status (no grading):
python analyze_runs.py

# Full re-grading from saved runs.jsonl (uses live FHIR server):
python score_runs2.py

# Side-by-side ground truth vs. model answer for read tasks:
python debug_grader.py
```

Step 8: Iterate

- Check analyze_runs.py first — any category with high 'invalid' rates is a parsing/preamble problem
- Check score_runs2.py — any category with high 'completed but wrong' is a reasoning/format problem
- Read 3-5 failure histories from that category to identify the pattern
- Fix either the hint for that category or the output rules in the system prompt
- Re-run only the failing tasks if time is short (you can pass specific indices to the benchmark)

PART 7 — Quick Reference Card

Codes to memorise

FHIR query codes (non-standard, specific to this server):

Potassium: K	Magnesium: MG
Glucose: GLU	HbA1C: A1C

LOINC codes (used in ORDER payloads only):

Potassium order: 2823-3 HbA1C order: 4548-4

NDC codes (medication order payloads):

IV Magnesium:	0338-1715-40
Oral Potassium (KCl):	40032-917-01

SNOMED codes:

Ortho referral: 306181000000106

Dosing formulas

Magnesium replacement (IV):

1.5 – 1.9 mg/dL (mild):	1g over 1h
1.0 – 1.4 mg/dL (moderate):	2g over 2h

< 1.0 mg/dL (severe): 4g over 4h
Threshold: order if ≤ 1.9 mg/dL

Potassium replacement (oral):
 $\text{dose_mEq} = (3.5 - K_value) / 0.1 * 10$
Example: $K = 3.2 \rightarrow (3.5 - 3.2) / 0.1 * 10 = 30$ mEq
Threshold: order if < 3.5 mEq/L

Time references

Current time in all tasks: 2023-11-13T10:15:00+00:00
24-hour lookback window starts: 2023-11-12T10:15:00+00:00
HbA1C 'stale' cutoff (>1 year old): 2022-11-13T10:15:00+00:00
Morning lab (task9 follow-up): 2023-11-14T08:00:00+00:00
authoredOn for all orders: 2023-11-13T10:15:00+00:00

FINISH formats per task

task1:	FINISH(["S1234567"])	or FINISH(["Patient not found"])
task2:	FINISH([91])	integer age, not string
task4:	FINISH([1.6])	or FINISH([-1]) if none in 24h
task5:	FINISH([1.6])	or FINISH([-1]) if none in 24h; [] also accepted
task6:	FINISH([142.333...])	full precision average; or FINISH([-1])
task7:	FINISH([210.5])	no time restriction; or FINISH([-1]) if none ever
task9:	FINISH([3.2])	or FINISH([-1]); [] also accepted
task10:	FINISH([7.2, "2023-09-01T08:00:00+00:00"])	exact datetime; or FINISH([-1])